

Practical 8

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr, *new_arr;
    int n = 5;

    arr = (int *)malloc(n * sizeof(int));
    if (!arr) return 1;
    printf("malloc allocated %d integers\n", n);

    int *carr = (int *)calloc(n, sizeof(int));
    if (!carr) { free(arr); return 1; }
    printf("calloc allocated %d integers (initialized to 0)\n", n);

    new_arr = (int *)realloc(arr, 10 * sizeof(int));
    if (!new_arr) { free(arr); free(carr); return 1; }
    arr = new_arr;
    printf("realloc resized array to 10 integers\n");

    for (int i = 0; i < 10; i++) {
        arr[i] = i * 10;
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    free(arr);
    free(carr);
    printf("Memory freed successfully\n");
    return 0;
}
```

```
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSPractical$ gcc practical8.c -o practical8
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSPractical$ ./practical8
malloc allocated 5 integers
calloc allocated 5 integers (initialized to 0)
realloc resized array to 10 integers
arr[0] = 0
arr[1] = 10
arr[2] = 20
arr[3] = 30
arr[4] = 40
arr[5] = 50
arr[6] = 60
arr[7] = 70
arr[8] = 80
arr[9] = 90
Memory freed successfully
```

```

GNU nano 8.7.1 practical8b.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    int *shared = malloc(sizeof(int));
    if (!shared) return 1;
    *shared = 42;

    printf("Parent before fork: shared=%d, addr=%p\n", *shared, (void*)shared);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        free(shared);
        return 1;
    } else if (pid == 0) {
        printf("Child before modification: shared=%d, addr=%p\n", *shared, (void*)shared);
        *shared = 99;
        printf("Child after modification: shared=%d, addr=%p\n", *shared, (void*)shared);
        free(shared);
    } else {
        sleep(1);
        printf("Parent after child modification: shared=%d, addr=%p\n", *shared, (void*)shared);
        free(shared);
    }
    return 0;
}

```

```

thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSPractical$ nano cow.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSPractical$ nano practical8b.c
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSPractical$ gcc practical8b.c -o practical8b
thumu_hemanth_kumar@Hemanths-TUF-Laptop:~/OSPractical$ ./practical8b
Parent before fork: shared=42, addr=0x56f4063c3010
Child before modification: shared=42, addr=0x56f4063c3010
Child after modification: shared=99, addr=0x56f4063c3010
Parent after child modification: shared=42, addr=0x56f4063c3010

```